12 Developing reactive APIs

This chapter covers

- Using Spring WebFlux
- Writing and testing reactive controllers and clients
- Consuming REST APIs
- Securing reactive web applications

Now that you've had a good introduction to reactive programming and Project Reactor, you're ready to start applying those techniques in your Spring applications. In this chapter, we're going to revisit some of the controllers you wrote in chapter 7 to take advantage of Spring's reactive programming model.

More specifically, we're going to take a look at Spring's reactive web framework—Spring WebFlux. As you'll quickly discover, Spring WebFlux is remarkably similar to Spring MVC, making it easy to apply, along with what you already know about building REST APIs in Spring.

12.1 Working with Spring WebFlux

Typical servlet web frameworks, such as Spring MVC, are blocking and multithreaded in nature, using a single thread per connection. As requests are handled, a worker thread is pulled from a thread pool to process the request. Meanwhile, the request thread is blocked until it's notified by the worker thread that it's finished.

Consequently, blocking web frameworks won't scale effectively under heavy request volume. Latency in slow worker threads makes things even worse because it'll take longer for the worker thread to be returned to the pool, ready to handle another request. In some use cases, this arrangement is perfectly acceptable. In fact, this is largely how most web applications have been developed for well over a decade. But times are changing.

The clients of those web applications have grown from people occasionally viewing websites to people frequently consuming content and using applications that coordinate

with HTTP APIs. And these days, the so-called *Internet of Things* (where humans aren't even involved) yields cars, jet engines, and other nontraditional clients constantly exchanging data with web APIs. With an increasing number of clients consuming web applications, scalability is more important than ever.

Asynchronous web frameworks, in contrast, achieve higher scalability with fewer threads—generally one per CPU core. By applying a technique known as *event looping* (as illustrated in figure 12.1), these frameworks are able to handle many requests per thread, making the per-connection cost more economical.

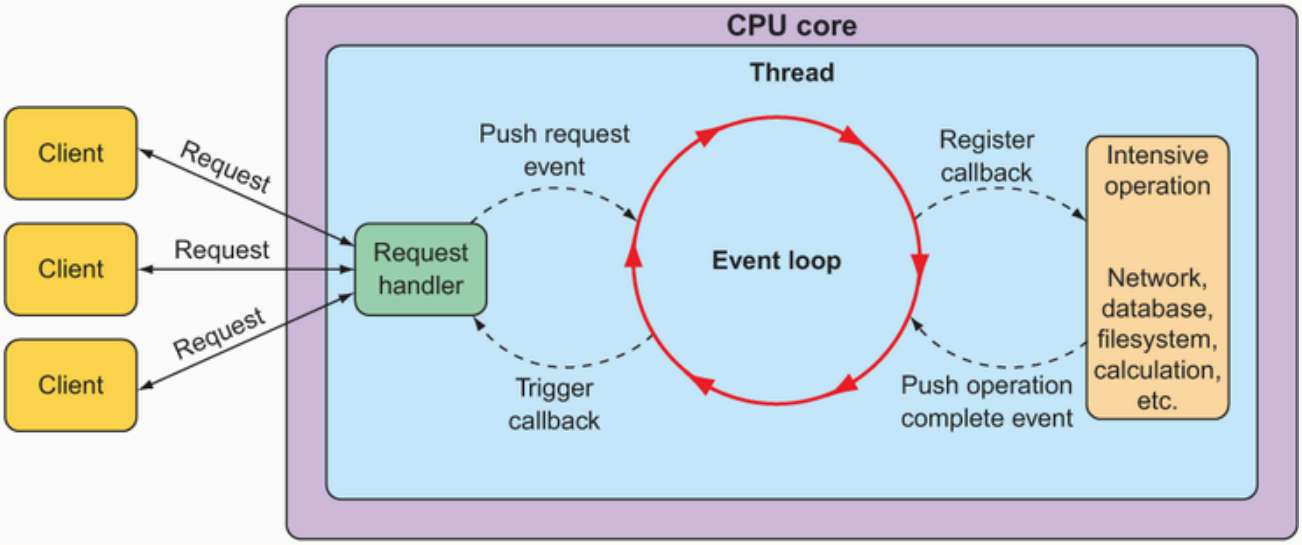


Figure 12.1 Asynchronous web frameworks apply event looping to handle more requests with fewer threads.

In an event loop, everything is handled as an event, including requests and callbacks from intensive operations like database and network operations. When a costly operation is needed, the event loop registers a callback for that operation to be performed in parallel, while it moves on to handle other events.

When the operation is complete, it's treated as an event by the event loop, the same as requests. As a result, asynchronous web frameworks are able to scale better under heavy request volume with fewer threads, resulting in reduced overhead for thread management.

Spring offers a nonblocking, asynchronous web framework based largely on its Project Reactor to address the need for greater scalability in web applications and APIs. Let's take a look at Spring WebFlux—a reactive web framework for Spring.

12.1.1 Introducing Spring WebFlux

As the Spring team was considering how to add a reactive programming model to the web layer, it quickly became apparent that it would be difficult to do so without a great

deal of work in Spring MVC. That would involve branching code to decide whether or not to handle requests reactively. In essence, the result would be two web frameworks packaged as one, with `if` statements to separate the reactive from the nonreactive.

Instead of trying to shoehorn a reactive programming model into Spring MVC, the Spring team decided to create a separate reactive web framework, borrowing as much from Spring MVC as possible. Spring WebFlux is the result. Figure 12.2 illustrates the complete web development stack available in Spring.

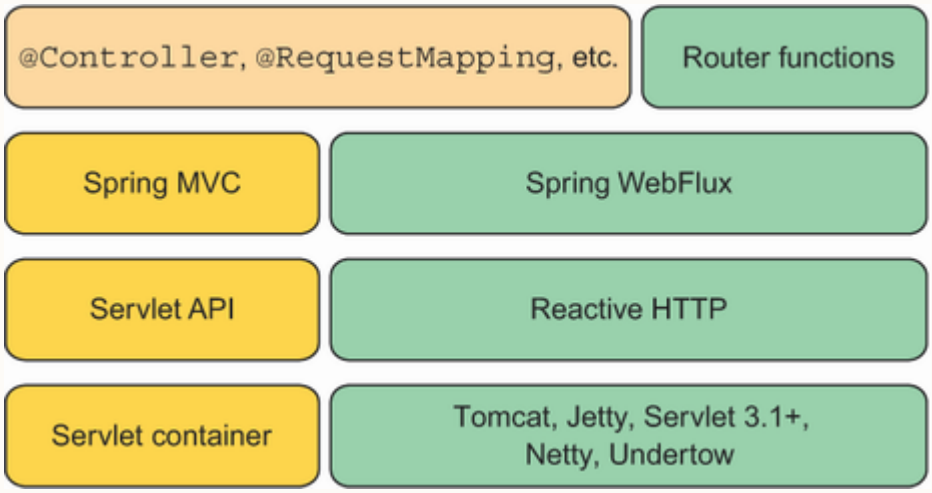


Figure 12.2 Spring supports reactive web applications with a new web framework called WebFlux, which is a sibling to Spring MVC and shares many of its core components.

On the left side of figure 12.2, you see the Spring MVC stack that was introduced in version 2.5 of the Spring Framework. Spring MVC (covered in chapters 2 and 7) sits atop the Java Servlet API, which requires a servlet container (such as Tomcat) to execute on.

By contrast, Spring WebFlux (on the right side) doesn't have ties to the Servlet API, so it builds on top of a Reactive HTTP API, which is a reactive approximation of the same functionality provided by the Servlet API. And because Spring WebFlux isn't coupled to the Servlet API, it doesn't require a servlet container to run on. Instead, it can run on any nonblocking web container including Netty, Undertow, Tomcat, Jetty, or any Servlet 3.1 or higher container.

What's most noteworthy about figure 12.2 is the top-left box, which represents the components that are common between Spring MVC and Spring WebFlux, primarily the annotations used to define controllers. Because Spring MVC and Spring WebFlux share the same annotations, Spring WebFlux is, in many ways, indistinguishable from Spring MVC.

The box in the top-right corner represents an alternative programming model that defines controllers with a functional programming paradigm instead of using annotations. We'll talk more about Spring's functional web programming model in section 12.2.

The most significant difference between Spring MVC and Spring WebFlux boils down to which dependency you add to your build. When working with Spring WebFlux, you'll need to add the Spring Boot WebFlux starter dependency instead of the standard web starter (e.g., `spring-boot-starter-web`). In the project's pom.xml file, it looks like this:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

NOTE As with most of Spring Boot's starter dependencies, this starter can also be added to a project by checking the Reactive Web check box in the Initializr.

An interesting side effect of using WebFlux instead of Spring MVC is that the default embedded server for WebFlux is Netty instead of Tomcat. Netty is one of a handful of asynchronous, event-driven servers and is a natural fit for a reactive web framework like Spring WebFlux.

Aside from using a different starter dependency, Spring WebFlux controller methods usually accept and return reactive types, like `Mono` and `Flux`, instead of domain types and collections. Spring WebFlux controllers can also deal with RxJava types like `Observable`, `Single`, and `Completable`.

REACTIVE SPRING MVC?

Although Spring WebFlux controllers typically return `Mono` and `Flux`, that doesn't mean that Spring MVC doesn't get to have some fun with reactive types. Spring MVC controller methods can also return a `Mono` or `Flux`, if you'd like.

The difference is in how those types are used. Whereas Spring WebFlux is a truly reactive web framework, allowing for requests to be handled in an event loop, Spring MVC is servlet-based, relying on multithreading to handle multiple requests.

Let's put Spring WebFlux to work by rewriting some of Taco Cloud's API controllers to take advantage of Spring WebFlux.

2.1.2 Writing reactive controllers

You may recall that in chapter 7, you created a few controllers for Taco Cloud's REST API. Those controllers had request-handling methods that dealt with input and output in terms of domain types (such as `TacoOrder` and `Taco`) or collections of those domain

types. As a reminder, consider the following snippet from `TacoController` that you wrote back in chapter 7:

```
@RestController
@RequestMapping(path="/api/tacos",
                produces="application/json")
@CrossOrigin(origins="*")
public class TacoController {

...

  @GetMapping(params="recent")
  public Iterable<Taco> recentTacos() {
    PageRequest page = PageRequest.of(
          0, 12, Sort.by("createdAt").descending());
    return tacoRepo.findAll(page).getContent();
  }

...

}
```

As written, the `recentTacos()` controller handles HTTP `GET` requests for /api/ tacos?recent to return a list of recently created tacos. More specifically, it returns an `Iterable` of type `Taco`. That's primarily because that's what's returned from the repository's `findAll()` method, or, more accurately, from the `getContent()` method on the `Page` object returned from `findAll()`.

That works fine, but `Iterable` isn't a reactive type. You won't be able to apply any reactive operations on it, nor can you let the framework take advantage of it as a reactive type to split any work over multiple threads. What you'd like is for `recentTacos()` to return a `Flux<Taco>`.

A simple but somewhat limited option here is to rewrite `recentTacos()` to convert the `Iterable` to a `Flux`. And, while you're at it, you can do away with the paging code and replace it with a call to `take()` on the `Flux` as follows:

```
@GetMapping(params="recent")
public Flux<Taco> recentTacos() {
  return Flux.fromIterable(tacoRepo.findAll()).take(12);
}
```

Using `Flux.fromIterable()`, you convert the `Iterable<Taco>` to a `Flux<Taco>`. And now that you're working with a `Flux`, you can use the `take()` operation to limit the returned `Flux` to 12 `Taco` objects at most. Not only is the code simpler, it also deals with a reactive `Flux` rather than a plain `Iterable`.

Writing reactive code has been a winning move so far. But it would be even better if the repository gave you a `Flux` to start with so that you wouldn't need to do the conversion. If that were the case, then `recentTacos()` could be written to look like this:

```
@GetMapping(params="recent")
public Flux<Taco> recentTacos() {
  return tacoRepo.findAll().take(12);
}
```

That's even better! Ideally, a reactive controller will be the tip of a stack that's reactive end to end, including controllers, repositories, the database, and any services that may sit in between. Such an end-to-end reactive stack is illustrated in figure 12.3.

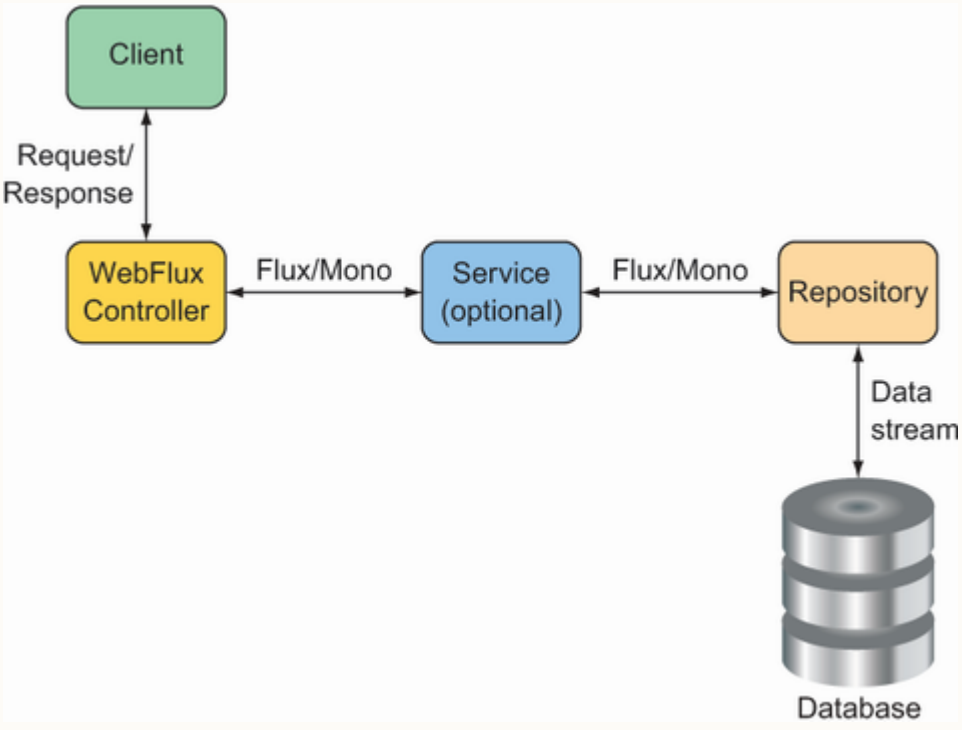


Figure 12.3 To maximize the benefit of a reactive web framework, it should be part of a full end-to-end reactive stack.

Such an end-to-end stack requires that the repository be written to return a `Flux` instead of an `Iterable`. We'll look into writing reactive repositories in the next chapter, but here's a sneak peek at what a reactive `TacoRepository` might look like:

```
package tacos.data;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;
import tacos.Taco;

public interface TacoRepository
      extends ReactiveCrudRepository<Taco, Long> {
}
```

What's most important to note at this point, however, is that aside from working with a `Flux` instead of an `Iterable`, as well as how you obtain that `Flux`, the programming model for defining a reactive WebFlux controller is no different than for a nonreactive Spring MVC controller. Both are annotated with `@RestController` and a high-level `@RequestMapping` at the class level. And both have request-handling functions that are annotated with `@GetMapping` at the method level. It's truly a matter of what type the handler methods return.

Another important observation to make is that although you're getting a `Flux<Taco>` back from the repository, you can return it without calling `subscribe()`. Indeed, the framework will call `subscribe()` for you. This means that when a request for /api/ tacos?recent is handled, the `recentTacos()` method will be called and will return before the data is even fetched from the database!

RETURNING SINGLE VALUES

As another example, consider the following `tacoById()` method from the `TacoController` as it was written in chapter 7:

```
@GetMapping("/{id}")
public Taco tacoById(@PathVariable("id") Long id) {
  Optional<Taco> optTaco = tacoRepo.findById(id);
  if (optTaco.isPresent()) {
    return optTaco.get();
  }
  return null;
}
```

Here, this method handles `GET` requests for /tacos/{id} and returns a single `Taco` object. Because the repository's `findById()` returns an `Optional`, you also had to write some clunky code to deal with that. But suppose for a minute that the `findById()` re-

turns a `Mono<Taco>` instead of an `Optional<Taco>`. In that case, you can rewrite the controller's `tacoById()` to look like this:

```
@GetMapping("/{id}")
public Mono<Taco> tacoById(@PathVariable("id") Long id) {
  return tacoRepo.findById(id);
}
```

Wow! That's a lot simpler. What's more important, however, is that by returning a `Mono<Taco>` instead of a `Taco`, you're enabling Spring WebFlux to handle the response in a reactive manner. Consequently, your API will scale better in response to heavy loads.

WORKING WITH RXJAVA TYPES

It's worth pointing out that although Reactor types like `Flux` and `Mono` are a natural choice when working with Spring WebFlux, you can also choose to work with RxJava types like `Observable` and `Single`. For example, suppose there's a service sitting between `TacoController` and the backend repository that deals in terms of RxJava types. In that case, you might write the `recentTacos()` method like this:

```
@GetMapping(params = "recent")
public Observable<Taco> recentTacos() {
  return tacoService.getRecentTacos();
}
```

Similarly, the `tacoById()` method could be written to deal with an RxJava `Single` rather than a `Mono`, as shown next:

```
@GetMapping("/{id}")
public Single<Taco> tacoById(@PathVariable("id") Long id) {
  return tacoService.lookupTaco(id);
}
```

In addition, Spring WebFlux controller methods can also return RxJava's `Completable`, which is equivalent to a `Mono<Void>` in Reactor. WebFlux can also return RxJava's `Flowable` as an alternative to `Observable` or Reactor's `Flux`.

HANDLING INPUT REACTIVELY

So far, we've concerned ourselves only with what reactive types the controller methods return. But with Spring WebFlux, you can also accept a `Mono` or a `Flux` as an input to a handler method. To demonstrate, consider the original implementation of `postTaco()` from `TacoController`, shown here:

```
@PostMapping(consumes="application/json")
@ResponseStatus(HttpStatus.CREATED)
public Taco postTaco(@RequestBody Taco taco) {
  return tacoRepo.save(taco);
}
```

As originally written, `postTaco()` not only returns a simple `Taco` object but also accepts a `Taco` object that's bound to the content in the body of the request. This means that `postTaco()` can't be invoked until the request payload has been fully resolved and used to instantiate a `Taco` object. It also means `postTaco()` can't return until the blocking call to the repository's `save()` method returns. In short, the request is blocked twice: as it enters `postTaco()` and again, inside of `postTaco()`. But by applying a little reactive coding to `postTaco()`, shown next, you can make it a fully nonblocking, request-handling method:

```
@PostMapping(consumes = "application/json")
@ResponseStatus(HttpStatus.CREATED)
public Mono<Taco> postTaco(@RequestBody Mono<Taco> tacoMono) {
  return tacoRepo.saveAll(tacoMono).next();
}
```

Here, `postTaco()` accepts a `Mono<Taco>` and calls the repository's `saveAll()` method, which accepts any implementation of Reactive Streams `Publisher`, including `Mono` or `Flux`. The `saveAll()` method returns a `Flux<Taco>`, but because you started with a `Mono`, you know there's at most one `Taco` that will be published by the `Flux`. You can therefore call `next()` to obtain a `Mono<Taco>` that will return from `postTaco()`.

By accepting a `Mono<Taco>` as input, the method is invoked immediately without waiting for the `Taco` to be resolved from the request body. And because the repository is also reactive, it'll accept a `Mono` and immediately return a `Flux<Taco>`, from which you call `next()` and return the resulting `Mono<Taco>` . . . all before the request is even processed!

Alternatively, you could also implement `postTaco()` like this:

```
@PostMapping(consumes = "application/json")
@ResponseStatus(HttpStatus.CREATED)
public Mono<Taco> postTaco(@RequestBody Mono<Taco> tacoMono) {
  return tacoMono.flatMap(tacoRepo::save);
}
```

This approach flips things around so that the `tacoMono` is the driver of the action. The `Taco` contained within `tacoMono` is handed to the repository's `save()` method via `flatMap()`, resulting in a new `Mono<Taco>` that is returned.

Either way works well, and there are probably several other ways that you could write `postTaco()`. Choose whichever way works best and makes the most sense to you.

Spring WebFlux is a fantastic alternative to Spring MVC, offering the option of writing reactive web applications using the same development model as Spring MVC. But Spring has another new trick up its sleeve. Let's take a look at how to create reactive APIs using Spring's functional programming style.

12.2 Defining functional request handlers

Spring MVC's annotation-based programming model has been around since Spring 2.5 and is widely popular. It comes with a few downsides, however.

First, any annotation-based programming involves a split in the definition of *what* the annotation is supposed to do and *how* it's supposed to do it. Annotations themselves define the what; the how is defined elsewhere in the framework code. This division complicates the programming model when it comes to any sort of customization or extension because such changes require working in code that's external to the annotation. Moreover, debugging such code is tricky because you can't set a breakpoint on an annotation.

Also, as Spring continues to grow in popularity, developers new to Spring from other languages and frameworks may find annotation-based Spring MVC (and WebFlux) quite unlike what they already know. As an alternative to WebFlux, Spring offers a functional programming model for defining reactive APIs.

This new programming model is used more like a library and less like a framework, letting you map requests to handler code without annotations. Writing an API using

Spring's functional programming model involves the following four primary types:

- `RequestPredicate` —Declares the kind(s) of requests that will be handled
- `RouterFunction` —Declares how a matching request should be routed to the handler code
- `ServerRequest` —Represents an HTTP request, including access to header and body information
- `ServerResponse` —Represents an HTTP response, including header and body information

As a simple example that pulls all of these types together, consider the following Hello World example:

```
package hello;

import static org.springframework.web
            .reactive.function.server.RequestPredicates.GET;
import static org.springframework.web
            .reactive.function.server.RouterFunctions.route;
import static org.springframework.web
            .reactive.function.server.ServerResponse.ok;
import static reactor.core.publisher.Mono.just;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.server.RouterFunction;

@Configuration
public class RouterFunctionConfig {

  @Bean
  public RouterFunction<?> helloRouterFunction() {
    return route(GET("/hello"),
        request -> ok().body(just("Hello World!"), String.class))
      ;
  }

}
```

The first thing to notice is that you've chosen to statically import a few helper classes that you can use to create the aforementioned functional types. You've also statically imported `Mono` to keep the rest of the code easier to read and understand.

In this `@Configuration` class, you have a single `@Bean` method of type `RouterFunction<?>`. As mentioned, a `RouterFunction` declares mappings between one or more `RequestPredicate` objects and the functions that will handle the matching request(s).

The `route()` method from `RouterFunctions` accepts two parameters: a `RequestPredicate` and a function to handle matching requests. In this case, the `GET()` method from `RequestPredicates` declares a `RequestPredicate` that matches HTTP `GET` requests for the /hello path.

As for the handler function, it's written as a lambda, although it can also be a method reference. Although it isn't explicitly declared, the handler lambda accepts a `ServerRequest` as a parameter. It returns a `ServerResponse` using `ok()` from `ServerResponse` and `body()` from `BodyBuilder`, which was returned from `ok()`. This was done to create a response with an HTTP 200 (OK) status code and a body payload that says `"Hello World!"`

As written, the `helloRouterFunction()` method declares a `RouterFunction` that handles only a single kind of request. But if you need to handle a different kind of request, you don't have to write another `@Bean` method, although you can. You only need to call `andRoute()` to declare another `RequestPredicate` to function mapping. For example, here's how you might add another handler for `GET` requests for /bye:

```
@Bean
public RouterFunction<?> helloRouterFunction() {
  return route(GET("/hello"),
     request -> ok().body(just("Hello World!"), String.class))
     .andRoute(GET("/bye"),
     request -> ok().body(just("See ya!"), String.class))
  ;
}
```

Hello World samples are fine for dipping your toes into something new. But let's amp it up a bit and see how to use Spring's functional web programming model to handle requests that resemble real-world scenarios.

To demonstrate how the functional programming model might be used in a real-world application, let's reinvent the functionality of `TacoController` in the functional style. The following configuration class is a functional analog to `TacoController`:

```
package tacos.web.api;

import static org.springframework.web.reactive.function.server
.RequestPredicates.GET;
import static org.springframework.web.reactive.function.server
.RequestPredicates.POST;
import static org.springframework.web.reactive.function.server
.RequestPredicates.queryParam;
import static org.springframework.web.reactive.function.server
.RouterFunctions.route;

import java.net.URI;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.reactive.function.server.RouterFunction;
import org.springframework.web.reactive.function.server.ServerRequest;
import org.springframework.web.reactive.function.server.ServerResponse;

import reactor.core.publisher.Mono;
import tacos.Taco;
import tacos.data.TacoRepository;
@Configuration
public class RouterFunctionConfig {

  @Autowired
  private TacoRepository tacoRepo;

  @Bean
  public RouterFunction<?> routerFunction() {
    return route(GET("/api/tacos").
            and(queryParam("recent", t->t != null )),
            this::recents)
        .andRoute(POST("/api/tacos"), this::postTaco);
  }

  public Mono<ServerResponse> recents(ServerRequest request) {
    return ServerResponse.ok()
        .body(tacoRepo.findAll().take(12), Taco.class);
  }

  public Mono<ServerResponse> postTaco(ServerRequest request) {
    return request.bodyToMono(Taco.class)
        .flatMap(taco -> tacoRepo.save(taco))
        .flatMap(savedTaco -> {
```

```
          return ServerResponse
              .created(URI.create(
                  "http://localhost:8080/api/tacos/" +
                  savedTaco.getId()))
              .body(savedTaco, Taco.class);
        });
    }
  }
```

As you can see, the `routerFunction()` method declares a `RouterFunction<?>` bean, like the Hello World example. But it differs in what types of requests are handled and how they're handled. In this case, the `RouterFunction` is created to handle `GET` requests for /api/tacos?recent and `POST` requests for /api/tacos.

What stands out even more is that the routes are handled by method references. Lambdas are great when the behavior behind a `RouterFunction` is relatively simple and brief. In many cases, however, it's better to extract that functionality into a separate method (or even into a separate method in a separate class) to maintain code readability.

For your needs, `GET` requests for /api/tacos?recent will be handled by the `recents()` method. It uses the injected `TacoRepository` to fetch a `Flux<Taco>`, from which it takes 12 items. It then wraps the `Flux<Taco>` in a `Mono<ServerResponse>` so that we can ensure that the response has an HTTP 200 (OK) status by calling `ok()` on the `ServerResponse`. It's important to understand that even though up to 12 tacos are returned, there is only one server response—that's why it is returned in a `Mono` and not a `Flux`. Internally, Spring will still stream the `Flux<Taco>` to the client as a `Flux`.

Meanwhile, `POST` requests for /api/tacos are handled by the `postTaco()` method, which extracts a `Mono<Taco>` from the body of the incoming `ServerRequest`. The `postTaco()` method then uses a series of `flatMap()` operations to save that taco to the `TacoRepository` and create a `ServerResponse` with an HTTP 201 (CREATED) status code and the saved `Taco` object in the response body.

The `flatMap()` operations are used to ensure that at each step in the flow, the result of the mapping is wrapped in a `Mono`, starting with a `Mono<Taco>` after the first `flatMap()` and ultimately ending with a `Mono<ServerResponse>` that is returned from `postTaco()`.

12.3 Testing reactive controllers

When it comes to testing reactive controllers, Spring hasn't left us in the lurch. Indeed, Spring has introduced `WebTestClient`, a new test utility that makes it easy to write tests for reactive controllers written with Spring WebFlux. To see how to write tests with `WebTestClient`, let's start by using it to test the `recentTacos()` method from the `TacoController` that you wrote in section 12.1.2.

12.3.1 Testing GET requests

One thing we'd like to assert about the `recentTacos()` method is that if an HTTP `GET` request is issued for the path /api/tacos?recent, then the response will contain a JSON payload with no more than 12 tacos. The test class in the next listing is a good start.

Listing 12.1 Using `WebTestClient` to test `TacoController`

```
package tacos.web.api;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import java.util.ArrayList;
import java.util.List;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.http.MediaType;
import org.springframework.test.web.reactive.server.WebTestClient;
import reactor.core.publisher.Flux;
import reactor.core.publisher.Mono;
import tacos.Ingredient;
import tacos.Ingredient.Type;
import tacos.Taco;
import tacos.data.TacoRepository;

public class TacoControllerTest {

  @Test
  public void shouldReturnRecentTacos() {
    Taco[] tacos = {
        testTaco(1L), testTaco(2L),
        testTaco(3L), testTaco(4L),                    (1)
        testTaco(5L), testTaco(6L),
        testTaco(7L), testTaco(8L),
        testTaco(9L), testTaco(10L),
        testTaco(11L), testTaco(12L),
        testTaco(13L), testTaco(14L),
```

```
      testTaco(15L), testTaco(16L)};
    Flux<Taco> tacoFlux = Flux.just(tacos);

    TacoRepository tacoRepo = Mockito.mock(TacoRepository.class);
    when(tacoRepo.findAll()).thenReturn(tacoFlux);                ❷

    WebTestClient testClient = WebTestClient.bindToController(
        new TacoController(tacoRepo))
        .build();                                                 ❸

    testClient.get().uri("/api/tacos?recent")
      .exchange()                                                 ❹
      .expectStatus().isOk()                                      ❺
      .expectBody()
        .jsonPath("$").isArray()
        .jsonPath("$").isNotEmpty()
        .jsonPath("$[0].id").isEqualTo(tacos[0].getId().toString())
        .jsonPath("$[0].name").isEqualTo("Taco 1")
        .jsonPath("$[1].id").isEqualTo(tacos[1].getId().toString())
        .jsonPath("$[1].name").isEqualTo("Taco 2")
        .jsonPath("$[11].id").isEqualTo(tacos[11].getId().toString())
        .jsonPath("$[11].name").isEqualTo("Taco 12")
        .jsonPath("$[12]").doesNotExist();
  }

  ...

}
```

❶ Creates some test data

❷ Mocks the TacoRepository

❸ Creates a WebTestClient

❹ Requests recent tacos

❺ Verifies the expected response

The first thing that the `shouldReturnRecentTacos()` method does is set up test data in the form of a `Flux<Taco>`. This `Flux` is then provided as the return value from the `findAll()` method of a mock `TacoRepository`.

With regard to the `Taco` objects that will be published by `Flux`, they're created with a utility method named `testTaco()` that, when given a number, produces a `Taco` object whose ID and name are based on that number. The `testTaco()` method is implemented as follows:

```
private Taco testTaco(Long number) {
  Taco taco = new Taco();
  taco.setId(number != null ? number.toString(): "TESTID");
  taco.setName("Taco " + number);
  List<Ingredient> ingredients = new ArrayList<>();
  ingredients.add(
      new Ingredient("INGA", "Ingredient A", Type.WRAP));
  ingredients.add(
      new Ingredient("INGB", "Ingredient B", Type.PROTEIN));
  taco.setIngredients(ingredients);
  return taco;
}
```

For the sake of simplicity, all test tacos will have the same two ingredients. But their ID and name will be determined by the given number.

Meanwhile, back in the `shouldReturnRecentTacos()` method, you instantiated a `TacoController`, injecting the mock `TacoRepository` into the constructor. The controller is given to `WebTestClient.bindToController()` to create an instance of `WebTestClient`.

With all of the setup complete, you're now ready to use `WebTestClient` to submit a `GET` request to /api/tacos?recent and verify that the response meets your expectations. Calling `get().uri("/api/tacos?recent")` describes the request you want to issue. Then a call to `exchange()` submits the request, which will be handled by the controller that `WebTestClient` is bound to—the `TacoController`.

Finally, you can affirm that the response is as expected. By calling `expectStatus()`, you assert that the response has an HTTP 200 (OK) status code. After that, you see several calls to `jsonPath()` that assert that the JSON in the response body has the values it should have. The final assertion checks that the 12th element (in a zero-based array) is nonexistent, because the result should never have more than 12 elements.

If the JSON returns are complex, with a lot of data or highly nested data, it can be tedious to use `jsonPath()`. In fact, I left out many of the calls to `jsonPath()` in listing 12.1 to conserve space. For those cases where it may be clumsy to use `jsonPath()`,

`WebTestClient` offers `json()`, which accepts a `String` parameter containing the JSON to compare the response against.

For example, suppose that you've created the complete response JSON in a file named recent-tacos.json and placed it in the classpath under the path /tacos. Then you can re-write the `WebTestClient` assertions to look like this:

```
ClassPathResource recentsResource =
    new ClassPathResource("/tacos/recent-tacos.json");
String recentsJson = StreamUtils.copyToString(
    recentsResource.getInputStream(), Charset.defaultCharset());

testClient.get().uri("/api/tacos?recent")
  .accept(MediaType.APPLICATION_JSON)
  .exchange()
  .expectStatus().isOk()
  .expectBody()
    .json(recentsJson);
```

Because `json()` accepts a `String`, you must first load the classpath resource into a `String`. Thankfully, Spring's `StreamUtils` makes this easy with `copyToString()`. The `String` that's returned from `copyToString()` will contain the entire JSON you expect in the response to your request. Giving it to the `json()` method ensures that the controller is producing the correct output.

Another option offered by `WebTestClient` allows you to compare the response body with a list of values. The `expectBodyList()` method accepts either a `Class` or a `ParameterizedTypeReference` indicating the type of elements in the list and returns a `ListBodySpec` object to make assertions against. Using `expectBodyList()`, you can re-write the test to use a subset of the same test data you used to create the mock `TacoRepository`, as shown here:

```
testClient.get().uri("/api/tacos?recent")
  .accept(MediaType.APPLICATION_JSON)
  .exchange()
  .expectStatus().isOk()
  .expectBodyList(Taco.class)
    .contains(Arrays.copyOf(tacos, 12));
```

Here you assert that the response body contains a list that has the same elements as the first 12 elements of the original `Taco` array you created at the beginning of the test

method.

2.3.2 Testing POST requests

`WebTestClient` can do more than just test `GET` requests against controllers. It can also be used to test any kind of HTTP method. Table 12.1 maps HTTP methods to `WebTestClient` methods.

Table 12.1 `WebTestClient` tests any kind of request against Spring WebFlux controllers.

HTTP method	WebTestClient method
GET	.get()
POST	.post()
PUT	.put()
PATCH	.patch()
DELETE	.delete()
HEAD	.head()

As an example of testing another HTTP method request against a Spring WebFlux controller, let's look at another test against `TacoController`. This time, you'll write a test of your API's taco creation endpoint by submitting a POST request to /api/tacos as follows:

```
@SuppressWarnings("unchecked")
@Test
public void shouldSaveATaco() {
  TacoRepository tacoRepo = Mockito.mock(
          TacoRepository.class);                          ❶

  WebTestClient testClient = WebTestClient.bindToController(        ❷
     new TacoController(tacoRepo)).build();

  Mono<Taco> unsavedTacoMono = Mono.just(testTaco(1L));
  Taco savedTaco = testTaco(1L);
  Flux<Taco> savedTacoMono = Flux.just(savedTaco);

  when(tacoRepo.saveAll(any(Mono.class))).thenReturn(savedTacoMono);  ❸
```

```
    testClient.post()                                    ❹
        .uri("/api/tacos")
        .contentType(MediaType.APPLICATION_JSON)
        .body(unsavedTacoMono, Taco.class)
        .exchange()
        .expectStatus().isCreated()                      ❺
        .expectBody(Taco.class)
        .isEqualTo(savedTaco);
  }
```

❶ Mocks the TacoRepository

❷ Creates a WebTestClient

❸ Sets up test data

❹ POSTs a taco

❺ Verifies the response

As with the previous test method, `shouldSaveATaco()` starts by mocking `TacoRepository`, building a `WebTestClient` that's bound to the controller, and setting up some test data. Then, it uses the `WebTestClient` to submit a `POST` request to /api/tacos, with a body of type `application/json` and a payload that's a JSON-serialized form of the `Taco` in the unsaved `Mono`. After performing `exchange()`, the test asserts that the response has an HTTP 201 (CREATED) status and a payload in the body equal to the saved `Taco` object.

2.3.3 Testing with a live server

The tests you've written so far have relied on a mock implementation of the Spring WebFlux framework so that a real server wouldn't be necessary. But you may need to test a WebFlux controller in the context of a server like Netty or Tomcat and maybe with a repository or other dependencies. That is to say, you may want to write an integration test.

To write a `WebTestClient` integration test, you start by annotating the test class with `@RunWith` and `@SpringBootTest` like any other Spring Boot integration test, as shown here:

```
package tacos;

import java.io.IOException;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.context.SpringBootTest.WebEnvironment;
import org.springframework.http.MediaType;
import org.springframework.test.context.junit.jupiter.SpringExtension;
import org.springframework.test.web.reactive.server.WebTestClient;

@ExtendWith(SpringExtension.class)
@SpringBootTest(webEnvironment=WebEnvironment.RANDOM_PORT)
public class TacoControllerWebTest {
  @Autowired
  private WebTestClient testClient;

}
```

By setting the `webEnvironment` attribute to `WebEnvironment.RANDOM_PORT`, you're asking Spring to start a running server listening on a randomly chosen port.[1]

You'll notice that you've also autowired a `WebTestClient` into the test class. This not only means that you'll no longer have to create one in your test methods but also that you won't need to specify a full URL when making requests. That's because the `WebTestClient` will be rigged to know which port the test server is running on. Now you can rewrite `shouldReturnRecentTacos()` as an integration test that uses the autowired `WebTestClient` as follows:

```
@Test
public void shouldReturnRecentTacos() throws IOException {
  testClient.get().uri("/api/tacos?recent")
    .accept(MediaType.APPLICATION_JSON).exchange()
    .expectStatus().isOk()
    .expectBody()
      .jsonPath("$").isArray()
      .jsonPath("$.length()").isEqualTo(3)
      .jsonPath("$[?(@.name == 'Carnivore')]").exists()
      .jsonPath("$[?(@.name == 'Bovine Bounty')]").exists()
      .jsonPath("$[?(@.name == 'Veg-Out')]").exists();
}
```

You've no doubt noticed that this new version of `shouldReturnRecentTacos()` has much less code. You no longer need to create a `WebTestClient` because you'll be making use of the autowired instance. And you don't have to mock `TacoRepository` because Spring will create an instance of `TacoController` and inject it with a real `TacoRepository`. In this new version of the test method, you use JSONPath expressions to verify values served from the database.

`WebTestClient` is useful when, in the course of a test, you need to consume the API exposed by a WebFlux controller. But what about when your application itself consumes some other API? Let's turn our attention to the client side of Spring's reactive web story and see how `WebClient` provides a REST client that deals in reactive types such as `Mono` and `Flux`.

12.4 Consuming REST APIs reactively

In chapter 8, you used `RestTemplate` to make client requests to the Taco Cloud API. `RestTemplate` is an old-timer, having been introduced in Spring version 3.0. In its time, it has been used to make countless requests on behalf of the applications that employ it.

But all of the methods provided by `RestTemplate` deal in nonreactive domain types and collections. This means that if you want to work with a response's data in a reactive way, you'll need to wrap it with a `Flux` or `Mono`. And if you already have a `Flux` or `Mono` and you want to send it in a `POST` or `PUT` request, then you'll need to extract the data into a nonreactive type before making the request.

It would be nice if there was a way to use `RestTemplate` natively with reactive types. Fear not. Spring offers `WebClient` as a reactive alternative to `RestTemplate`. `WebClient` lets you both send and receive reactive types when making requests to external APIs.

Using `WebClient` is quite different from using `RestTemplate`. Rather than having several methods to handle different kinds of requests, `WebClient` has a fluent builder-style interface that lets you describe and send requests. The general usage pattern for working with `WebClient` follows:

- Create an instance of `WebClient` (or inject a `WebClient` bean)
- Specify the HTTP method of the request to send
- Specify the URI and any headers that should be in the request
- Submit the request
- Consume the response

Let's look at several examples of `WebClient` in action, starting with how to use `WebClient` to send HTTP `GET` requests.

2.4.1 GETting resources

As an example of `WebClient` usage, suppose that you need to fetch an `Ingredient` object by its ID from the Taco Cloud API. Using `RestTemplate`, you might use the `getForObject()` method. But with `WebClient`, you build the request, retrieve a response, and then extract a `Mono` that publishes the `Ingredient` object, as shown here:

```
Mono<Ingredient> ingredient = WebClient.create()
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);

ingredient.subscribe(i -> { ... });
```

Here you create a new `WebClient` instance with `create()`. Then you use `get()` and `uri()` to define a `GET` request to http://localhost:8080/ingredients/{id}, where the {id} placeholder will be replaced by the value in `ingredientId`. The `retrieve()` method executes the request. Finally, a call to `bodyToMono()` extracts the response's body payload into a `Mono<Ingredient>` on which you can continue applying additional `Mono` operations.

To apply additional operations on the `Mono` returned from `bodyToMono()`, it's important to subscribe to it before the request will even be sent. Making requests that can return a collection of values is easy. For example, the following snippet of code fetches all ingredients:

```
Flux<Ingredient> ingredients = WebClient.create()
    .get()
    .uri("http://localhost:8080/ingredients")
    .retrieve()
    .bodyToFlux(Ingredient.class);

ingredients.subscribe(i -> { ... });
```

For the most part, fetching multiple items is the same as making a request for a single item. The big difference is that instead of using `bodyToMono()` to extract the response's

body into a `Mono`, you use `bodyToFlux()` to extract it into a `Flux`.

As with `bodyToMono()`, the `Flux` returned from `bodyToFlux()` hasn't yet been subscribed to. This allows additional operations (filters, maps, and so forth) to be applied to the `Flux` before the data starts flowing through it. Therefore, it's important to subscribe to the resulting `Flux`, or else the request will never even be sent.

MAKING REQUESTS WITH A BASE URI

You may find yourself using a common base URI for many different requests. In that case, it can be useful to create a `WebClient` bean with a base URI and inject it anywhere it's needed. Such a bean could be declared like this (in any `@Configuration`-annotated class):

```
@Bean
public WebClient webClient() {
  return WebClient.create("http://localhost:8080");
}
```

Then, anywhere you need to make requests using that base URI, the `WebClient` bean can be injected and used like this:

```
@Autowired
WebClient webClient;

public Mono<Ingredient> getIngredientById(String ingredientId) {
  Mono<Ingredient> ingredient = webClient
    .get()
    .uri("/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);

  ingredient.subscribe(i -> { ... });
}
```

Because the `WebClient` had already been created, you're able to get right to work by calling `get()`. As for the URI, you need to specify only the path relative to the base URI when calling `uri()`.

TIMING OUT ON LONG-RUNNING REQUESTS

One thing that you can count on is that networks aren't always reliable or as fast as you'd expect them to be. Or maybe a remote server is sluggish in handling a request. Ideally, a request to a remote service will return in a reasonable amount of time. But if not, it would be great if the client didn't get stuck waiting on a response for too long.

To avoid having your client requests held up by a sluggish network or service, you can use the `timeout()` method from `Flux` or `Mono` to put a limit on how long you'll wait for data to be published. As an example, consider how you might use `timeout()` when fetching ingredient data, as shown in the next code sample:

```
Flux<Ingredient> ingredients = webclient
    .get()
    .uri("/ingredients")
    .retrieve()
    .bodyToFlux(Ingredient.class);

ingredients
  .timeout(Duration.ofSeconds(1))
  .subscribe(
      i -> { ... },
      e -> {
        // handle timeout error
      });
```

As you can see, before subscribing to the `Flux`, you called `timeout()`, specifying a duration of 1 second. If the request can be fulfilled in less than 1 second, then there's no problem. But if the request is taking longer than 1 second, it'll time-out, and the error handler given as the second parameter to `subscribe()` is invoked.

2.4.2 Sending resources

Sending data with `WebClient` isn't much different from receiving data. As an example, let's say that you have a `Mono<Ingredient>` and want to send a `POST` request with the `Ingredient` that's published by the `Mono` to the URI with a relative path of /ingredients. All you must do is use the `post()` method instead of `get()` and specify that the `Mono` is to be used to populate the request body by calling `body()` as follows:

```
Mono<Ingredient> ingredientMono = Mono.just(
    new Ingredient("INGC", "Ingredient C", Ingredient.Type.VEGGIES));
```

```
Mono<Ingredient> result = webClient
  .post()
  .uri("/ingredients")
  .body(ingredientMono, Ingredient.class)
  .retrieve()
  .bodyToMono(Ingredient.class);

result.subscribe(i -> { ... });
```

If you don't have a `Mono` or `Flux` to send, but instead have the raw domain object on hand, you can use `bodyValue()`. For example, suppose that instead of a `Mono<Ingredient>`, you have an `Ingredient` that you want to send in the request body, as shown next:

```
Ingredient ingredient = ...;

Mono<Ingredient> result = webClient
  .post()
  .uri("/ingredients")
  .bodyValue(ingredient)
  .retrieve()
  .bodyToMono(Ingredient.class);

result.subscribe(i -> { ... });
```

Instead of a `POST` request, if you want to update an `Ingredient` with a `PUT` request, you call `put()` instead of `post()` and adjust the URI path accordingly, like so:

```
Mono<Void> result = webClient
  .put()
  .uri("/ingredients/{id}", ingredient.getId())
  .bodyValue(ingredient)
  .retrieve()
  .bodyToMono(Void.class);

result.subscribe();
```

`PUT` requests typically have empty response payloads, so you must ask `bodyToMono()` to return a `Mono` of type `Void`. On subscribing to that `Mono`, the request will be sent.

2.4.3 Deleting resources

`WebClient` also allows the removal of resources by way of its `delete()` method. For example, the following code deletes an ingredient for a given ID:

```
Mono<Void> result = webClient
  .delete()
  .uri("/ingredients/{id}", ingredientId)
  .retrieve()
  .bodyToMono(Void.class);

result.subscribe();
```

As with `PUT` requests, `DELETE` requests don't typically have a payload. Once again, you return and subscribe to a `Mono<Void>` to send the request.

2.4.4 Handling errors

All of the `WebClient` examples thus far have assumed a happy ending; there were no responses with 400-level or 500-level status codes. Should either kind of error statuses be returned, `WebClient` will log the failure and move on without incident.

If you need to handle such errors, then a call to `onStatus()` can be used to specify how various HTTP status codes should be dealt with. `onStatus()` accepts two functions: a predicate function, which is used to match the HTTP status, and a function that, given a `ClientResponse` object, returns a `Mono<Throwable>`.

To demonstrate how `onStatus()` can be used to create a custom error handler, consider the following use of `WebClient` that aims to fetch an ingredient given its ID:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);
```

As long as the value in `ingredientId` matches a known ingredient resource, then the resulting `Mono` will publish the `Ingredient` object when it's subscribed to. But what would happen if there were no matching ingredient?

When subscribing to a `Mono` or `Flux` that might end in an error, it's important to register an error consumer as well as a data consumer in the call to `subscribe()` as follows:

```
ingredientMono.subscribe(
    ingredient -> {
      // handle the ingredient data
      ...
    },
    error-> {
      // deal with the error
      ...
    });
```

If the ingredient resource is found, then the first lambda (the data consumer) given to `subscribe()` is invoked with the matching `Ingredient` object. But if it isn't found, then the request responds with a status code of HTTP 404 (NOT FOUND), which results in the second lambda (the error consumer) being given by default a `WebClientResponseException`.

The biggest problem with `WebClientResponseException` is that it's rather nonspecific as to what may have gone wrong to cause the `Mono` to fail. Its name suggests that there was an error in the response from a request made by `WebClient`, but you'll need to dig into `WebClientResponseException` to know what went wrong. And in any event, it would be nice if the exception given to the error consumer were more domain-specific instead of `WebClient`-specific.

By adding a custom error handler, you can provide code that translates a status code to a `Throwable` of your own choosing. Let's say that you want a failed request for an ingredient resource to cause the `Mono` to complete in error with an `UnknownIngredientException`. You can add the following call to `onStatus()` after the call to `retrieve()` to achieve that:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .onStatus(HttpStatus::is4xxClientError,
          response -> Mono.just(new UnknownIngredientException()))
    .bodyToMono(Ingredient.class);
```

The first argument in the `onStatus()` call is a predicate that's given an `HttpStatus` and returns `true` if the status code is one you want to handle. And if the status code matches, then the response will be returned to the function in the second argument to handle as it sees fit, ultimately returning a `Mono` of type `Throwable`.

In the example, if the status code is a 400-level status code (e.g., a client error), then a `Mono` will be returned with an `UnknownIngredientException`. This causes the `ingredientMono` to fail with that exception.

Note that `HttpStatus::is4xxClientError` is a method reference to the `is4xxClientError` method of `HttpStatus`. It's this method that will be invoked on the given `HttpStatus` object. If you want, you can use another method on `HttpStatus` as a method reference; or you can provide your own function in the form of a lambda or method reference that returns a `boolean`.

For example, you can get even more precise in your error handling, checking specifically for an HTTP 404 (NOT FOUND) status by changing the call to `onStatus()` to look like this:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .onStatus(status -> status == HttpStatus.NOT_FOUND,
          response -> Mono.just(new UnknownIngredientException()))
    .bodyToMono(Ingredient.class);
```

It's also worth noting that you can have as many calls to `onStatus()` as you need to handle any variety of HTTP status codes that might come back in the response.

2.4.5 Exchanging requests

Up to this point, you've used the `retrieve()` method to signify sending a request when working with `WebClient`. In those cases, the `retrieve()` method returned an object of type `ResponseSpec`, through which you were able to handle the response with calls to methods such as `onStatus()`, `bodyToFlux()`, and `bodyToMono()`. Working with `ResponseSpec` is fine for simple cases, but it's limited in a few ways. If you need access to the response's headers or cookie values, for example, then `ResponseSpec` isn't going to work for you.

When `ResponseSpec` comes up short, you can try calling `exchangeToMono()` or `exchangeToFlux()` instead of `retrieve()`. The `exchangeToMono()` method returns a `Mono` of type `ClientResponse`, on which you can apply reactive operations to inspect and use data from the entire response, including the payload, headers, and cookies. The `exchangeToFlux()` method works much the same way but returns a `Flux` of type `ClientResponse` for working with multiple data items in the response.

Before we look at what makes `exchangeToMono()` and `exchangeToFlux()` different from `retrieve()`, let's start by looking at how similar they are. The following snippet of code uses a `WebClient` and `exchangeToMono()` to fetch a single ingredient by the ingredient's ID:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .exchangeToMono(cr -> cr.bodyToMono(Ingredient.class));
```

This is roughly equivalent to the next example that uses `retrieve()`:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
    .retrieve()
    .bodyToMono(Ingredient.class);
```

In the `exchangeToMono()` example, rather than use the `ResponseSpec` object's `bodyToMono()` to get a `Mono<Ingredient>`, you get a `Mono<ClientResponse>` on which you can apply a flat-mapping function to map the `ClientResponse` to a `Mono<Ingredient>`, which is flattened into the resulting `Mono`.

Let's see what makes `exchangeToMono()` different from `retrieve()`. Let's suppose that the response from the request might include a header named `X_UNAVAILABLE` with a value of `true` to indicate that (for some reason) the ingredient in question is unavailable. And for the sake of discussion, suppose that if that header exists, you want the resulting `Mono` to be empty—to not return anything. You can achieve this scenario by adding another call to `flatMap()`, but now it's simpler with a `WebClient` call like this:

```
Mono<Ingredient> ingredientMono = webClient
    .get()
    .uri("http://localhost:8080/ingredients/{id}", ingredientId)
```

```
    .exchangeToMono(cr -> {
      if (cr.headers().header("X_UNAVAILABLE").contains("true")) {
        return Mono.empty();
      }
      return Mono.just(cr);
    })
    .flatMap(cr -> cr.bodyToMono(Ingredient.class));
```

The new `flatMap()` call inspects the given `ClientRequest` object's headers, looking for a header named `X_UNAVAILABLE` with a value of `true`. If found, it returns an empty `Mono`. Otherwise, it returns a new `Mono` that contains the `ClientResponse`. In either event, the `Mono` returned will be flattened into the `Mono` that the next `flatMap()` call will operate on.

12.5 Securing reactive web APIs

For as long as there has been Spring Security (and even before that, when it was known as Acegi Security), its web security model has been built around servlet filters. After all, it just makes sense. If you need to intercept a request bound for a servlet-based web framework to ensure that the requester has proper authority, a servlet filter is an obvious choice. But Spring WebFlux puts a kink into that approach.

When writing a web application with Spring WebFlux, there's no guarantee that servlets are even involved. In fact, a reactive web application is debatably more likely to be built on Netty or some other nonservlet server. Does this mean that the servlet filter–based Spring Security can't be used to secure Spring WebFlux applications?

It's true that using servlet filters isn't an option when securing a Spring WebFlux application. But Spring Security is still up to the task. Starting with version 5.0.0, you can use Spring Security to secure both servlet-based Spring MVC and reactive Spring WebFlux applications. It does this using Spring's `WebFilter`, a Spring-specific analog to servlet filters that doesn't demand dependence on the servlet API.

What's even more remarkable, though, is that the configuration model for reactive Spring Security isn't much different from what you saw in chapter 4. In fact, unlike Spring WebFlux, which has a separate dependency from Spring MVC, Spring Security comes as the same Spring Boot security starter, regardless of whether you intend to use it to secure a Spring MVC web application or one written with Spring WebFlux. As a reminder, here's what the security starter looks like:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

That said, a few small differences exist between Spring Security's reactive and nonreactive configuration models. It's worth taking a quick look at how the two configuration models compare.

12.5.1 Configuring reactive web security

As a reminder, configuring Spring Security to secure a Spring MVC web application typically involves creating a new configuration class that extends `WebSecurityConfigurerAdapter` and is annotated with `@EnableWebSecurity`. Such a configuration class would override a `configuration()` method to specify web security specifics such as what authorizations are required for certain request paths. The following simple Spring Security configuration class serves as a reminder of how to configure security for a nonreactive Spring MVC application:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {

  @Override
  protected void configure(HttpSecurity http) throws Exception {
    http
      .authorizeRequests()
        .antMatchers("/api/tacos", "/orders").hasAuthority("USER")
        .antMatchers("/**").permitAll();
  }

}
```

Now let's see what this same configuration might look like for a reactive Spring WebFlux application. The following listing shows a reactive security configuration class that's roughly equivalent to the simple security configuration from before.

Listing 12.2 Configuring Spring Security for a Spring WebFlux application

```
@Configuration
@EnableWebFluxSecurity
```

```
public class SecurityConfig {

  @Bean
  public SecurityWebFilterChain securityWebFilterChain(
                                ServerHttpSecurity http) {
    return http
        .authorizeExchange()
          .pathMatchers("/api/tacos", "/orders").hasAuthority("USER")
          .anyExchange().permitAll()
      .and()
        .build();
  }

}
```

As you can see, there's a lot that's familiar, though, at the same time, much is different. Rather than `@EnableWebSecurity`, this new configuration class is annotated with `@EnableWebFluxSecurity`. What's more, the configuration class doesn't extend `WebSecurityConfigurerAdapter` or any other base class whatsoever. Therefore, it also doesn't override any `configure()` methods.

In place of a `configure()` method, you declare a bean of type `SecurityWebFilterChain` with the `securityWebFilterChain()` method. The body of `securityWebFilterChain()` isn't much different from the previous configuration's `configure()` method, but there are some subtle changes.

Primarily, the configuration is declared using a given `ServerHttpSecurity` object instead of an `HttpSecurity` object. Using the given `ServerHttpSecurity`, you can call `authorizeExchange()`, which is roughly equivalent to `authorizeRequests()`, to declare request-level security.

NOTE `ServerHttpSecurity` is new to Spring Security 5 and is the reactive analog to `HttpSecurity`.

When matching paths, you can still use Ant-style wildcard paths, but do so with the `pathMatchers()` method instead of `antMatchers()`. And as a convenience, you no longer need to specify a catchall Ant-style path of /** because the `anyExchange()` returns the catchall you need.

Finally, because you're declaring the `SecurityWebFilterChain` as a bean instead of overriding a framework method, you must call the `build()` method to assemble all of the security rules into the `SecurityWebFilterChain` to be returned.

Aside from those small differences, configuring web security isn't that different for Spring WebFlux than for Spring MVC. But what about user details?

2.5.2 Configuring a reactive user details service

When extending `WebSecurityConfigurerAdapter`, you override one `configure()` method to declare web security rules and another `configure()` method to configure authentication logic, typically by defining a `UserDetails` object. As a reminder of what this looks like, consider the following overridden `configure()` method that uses an injected `UserRepository` object in an anonymous implementation of `UserDetailsService` to look up a user by username:

```
@Autowired
UserRepository userRepo;

@Override
protected void
    configure(AuthenticationManagerBuilder auth)
    throws Exception {
  auth
    .userDetailsService(new UserDetailsService() {
      @Override
      public UserDetails loadUserByUsername(String username)
                          throws UsernameNotFoundException {
        User user = userRepo.findByUsername(username)
        if (user == null) {
          throw new UsernameNotFoundException(
                    username " + not found")
        }
        return user.toUserDetails();
      }
    });
}
```

In this nonreactive configuration, you override the only method required by `UserDetailsService`: `loadUserByUsername()`. Inside of that method, you use the given `UserRepository` to look up the user by the given username. If the name isn't found, you throw a `UsernameNotFoundException`. But if it's found, then you call a helper method, `toUserDetails()`, to return the resulting `UserDetails` object.

In a reactive security configuration, you don't override a `configure()` method. Instead, you declare a `ReactiveUserDetailsService` bean. `ReactiveUserDetailsService` is the reac-

tive equivalent to `UserDetailsService`. Like `UserDetailsService`, `ReactiveUserDetailsService` requires implementation of only a single method. Specifically, the `findByUsername()` method returns a `Mono<UserDetails>` instead of a raw `UserDetails` object.

In the following example, the `ReactiveUserDetailsService` bean is declared to use a given `UserRepository`, which is presumed to be a reactive Spring Data repository (which we'll talk more about in the next chapter):

```
@Bean
public ReactiveUserDetailsService userDetailsService(
                                UserRepository userRepo) {
  return new ReactiveUserDetailsService() {
    @Override
    public Mono<UserDetails> findByUsername(String username) {
      return userRepo.findByUsername(username)
        .map(user -> {
          return user.toUserDetails();
        });
    }
  };
}
```

Here, a `Mono<UserDetails>` is returned as required, but the `UserRepository.findByUsername()` method returns a `Mono<User>`. Because it's a `Mono`, you can chain operations on it, such as a `map()` operation to map the `Mono<User>` to a `Mono<UserDetails>`.

In this case, the `map()` operation is applied with a lambda that calls the helper `toUserDetails()` method on the `User` object published by the `Mono`. This converts the `User` to a `UserDetails`. As a consequence, the `.map()` operation returns a `Mono<UserDetails>`, which is precisely what the `ReactiveUserDetailsService.findByUsername()` requires. If `findByUsername()` can't find a matching user, then the `Mono` returned will be empty, indicating no match and resulting in a failure to authenticate.

Summary

- Spring WebFlux offers a reactive web framework whose programming model mirrors that of Spring MVC and even shares many of the same annotations.

- Spring also offers a functional programming model as an alternative to Spring WebFlux's annotation-based programming model.
- Reactive controllers can be tested with `WebTestClient`.
- On the client side, Spring offers `WebClient`, a reactive analog to Spring's `RestTemplate`.
- Although WebFlux has some significant implications for the underlying mechanisms for securing a web application, Spring Security 5 supports reactive security with a programming model that isn't dramatically different from nonreactive Spring MVC applications.

---

[1] You could have also set `webEnvironment` to `WebEnvironment.DEFINED_PORT` and specified a port with the properties attribute, but that's generally inadvisable. Doing so opens the risk of a port clash with a concurrently running server.

Previous chapter

< 11 Introducing Reactor

Next chapter

13 Persisting data reactively >